PROJECT REPORT

VIRTUAL MOUSE USING HAND GESTURES

A touchless computer interaction system using computer vision

Prepared by	Madupath Hari Haran
Institution	KL University
Project folder	HandGestureControl
Primary file	main.py
Technology area	Python · Computer Vision · Human–Computer Interaction

This report describes the design, implementation, setup, and functional behaviour of a webcam-based virtual mouse controlled through hand gestures.

Abstract

The Virtual Mouse Using Hand Gestures project is a touchless computer interaction system. It uses a webcam to capture the user's hand, OpenCV to process video frames, MediaPipe Hands to locate hand landmarks, and PyAutoGUI to translate recognized gestures into operating-system mouse actions. NumPy is used for coordinate mapping and distance calculations.

The current implementation supports cursor movement, vertical scrolling, horizontal scrolling, a stop gesture, and a pinch-like click gesture formed by bringing the index and middle fingertips close together. The application runs continuously until the user presses the Q key in the camera window. The project demonstrates how real-time computer vision can provide an alternative to physical mouse input.

1. Introduction

A conventional computer mouse requires physical contact and may not be convenient in every situation. A gesture-controlled virtual mouse offers an alternative by interpreting hand movements captured through a camera. The goal of this project is to build a lightweight prototype that maps a small set of intuitive hand gestures to common mouse operations.

1.1 Problem statement

Design and implement a webcam-based application that detects a user's hand in real time and converts selected finger configurations and fingertip movements into mouse control commands.

1.2 Objectives

• Capture live video from the system webcam.

• Detect and track one hand using MediaPipe Hands.

• Count extended fingers, excluding the thumb, to select an interaction mode.

• Map the index fingertip position to screen coordinates for cursor movement.

• Use fingertip movement to perform vertical and horizontal scrolling.

• Detect proximity between the index and middle fingertips to trigger a mouse click.

• Provide a simple on-screen status label and a keyboard exit control.

2. Tools and Technologies

Technology	Purpose in the project
Python 3.12	Programming language and runtime environment.
OpenCV (opencv-contrib-python 4.11.0.86)	Webcam capture, frame conversion, image display, and text overlays.
MediaPipe 0.10.21	Hand detection and 21-point hand landmark tracking.
PyAutoGUI	Operating-system mouse movement, clicking, and scrolling.
NumPy 1.26.4	Coordinate interpolation and Euclidean distance calculation.
Virtual environment (venv)	Isolates project dependencies from global Python packages.

The project uses a Python virtual environment so the required packages can be installed and maintained independently. The tested package combination recorded during setup is Python 3.12, MediaPipe 0.10.21, NumPy 1.26.4, and OpenCV Contrib 4.11.0.86.

3. System Requirements

3.1 Hardware

• A computer running Windows with a working webcam.

• A display and sufficient processing capability for live video processing.

• A reasonably lit environment with the hand visible inside the camera frame.

3.2 Software

• Windows operating system and Python 3.12.

• Visual Studio Code or another Python-compatible editor (optional).

• Packages: opencv-contrib-python, mediapipe, pyautogui, and numpy.

4. System Design and Workflow

The application follows a frame-by-frame processing pipeline. The webcam supplies a frame; the frame is mirrored and converted from BGR to RGB; MediaPipe processes the RGB image and returns hand landmarks when a hand is detected. The program uses landmark positions to count extended fingers and measure fingertip distances or movement. It then issues the corresponding mouse action through PyAutoGUI and displays the camera frame.

1	**Webcam**	Capture a frame
2	**OpenCV**	Flip frame and convert BGR to RGB
3	**MediaPipe Hands**	Detect hand and return landmarks
4	**Gesture logic**	Count fingers and calculate movement/distance
5	**PyAutoGUI**	Move, click, or scroll the mouse
6	**OpenCV display**	Show frame and gesture status

4.1 Landmark-based finger counting

MediaPipe Hands provides 21 landmarks for a detected hand. The implementation checks the y-coordinate of the tips of the index, middle, ring, and little fingers (landmark indices 8, 12, 16, and 20) against the corresponding middle-joint landmarks (indices 6, 10, 14, and 18). In the mirrored image coordinate system, a fingertip with a smaller y-coordinate is treated as extended. The thumb is deliberately excluded from the finger count.

4.2 Coordinate mapping

The index fingertip's image coordinates are mapped to the screen dimensions using NumPy interpolation. This converts the camera-frame position into a corresponding screen position. The current implementation maps the full camera width and height directly to the full screen width and height.

5. Gesture Mapping

Gesture / condition	Program action	Implementation detail
1 extended finger	Move cursor	Index fingertip mapped to screen coordinates.
2 extended fingers	Vertical scroll	Change in index fingertip y-coordinate controls scroll direction and amount.
3 extended fingers	Horizontal scroll	Change in index fingertip x-coordinate controls horizontal scroll.

Gesture / condition	Program action	Implementation detail
0 extended fingers	Stop	Displays STOP; no cursor/scroll action is selected.
Index and middle tips close	Mouse click	Click when fingertip distance is below 30 pixels and cooldown has elapsed.
Press Q in camera window	Exit	Breaks the loop and releases camera/display resources.

The click check is evaluated independently of the finger-count branches. Therefore, bringing the index and middle fingertips within the threshold can trigger a click even while another gesture mode is active. A one-second cooldown is used to reduce repeated clicks.

6. Implementation Overview

6.1 Main configuration

```
pyautogui.FAILSAFE = False
pyautogui.PAUSE = 0

SCREEN_W, SCREEN_H = pyautogui.size()
CLICK_COOLDOWN = 1.0
SCROLL_SPEED = 2
DEADZONE = 5

cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)
```

The camera is opened using device index 0 and the DirectShow backend. The application configures a one-second click cooldown, a scroll multiplier, and a small movement deadzone.

6.2 Hand tracking setup

```
mp_hands = mp.solutions.hands
hands = mp_hands.Hands(
    max_num_hands=1,
    min_detection_confidence=0.7,
    min_tracking_confidence=0.7
)
```

The detector is configured to track a maximum of one hand. Both detection and tracking confidence thresholds are set to 0.7.

6.3 Main processing loop

The loop reads frames until the camera read fails or the user presses Q. Each valid frame is mirrored, converted to RGB, and passed to the hand detector. When landmarks are available, the program calculates the finger count, fingertip coordinates, and distance between the index and middle fingertips. The appropriate PyAutoGUI action is then executed and OpenCV displays the resulting frame.

```
while True:
    ret, frame = cap.read()
    if not ret:
        break

    frame = cv2.flip(frame, 1)
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    result = hands.process(rgb)

    # Count extended fingers and read fingertip positions
    # Select movement / scroll behaviour
    # Check index-middle fingertip distance for click

    cv2.imshow("Gesture Control", frame)
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

cap.release()
```

```
cv2.destroyAllWindows()
```

7. Setup and Execution

The following commands reproduce the working setup in Windows PowerShell. Run them from the HandGestureControl project directory.

```
# Create the environment with Python 3.12
py -3.12 -m venv venv

# Activate it
.\venv\Scripts\Activate.ps1

# Install compatible packages
python -m pip install --upgrade pip
python -m pip install mediapipe==0.10.21 numpy==1.26.4
python -m pip install opencv-contrib-python==4.11.0.86 pyautogui

# Verify imports
python -c "import cv2, mediapipe, pyautogui, numpy; print('ALL LIBRARIES WORKING!')"

# Run the project
python main.py
```

Only one OpenCV distribution should be installed in this environment. OpenCV's standard and contrib packages share the cv2 namespace; installing both can leave conflicting or incomplete files. The working setup uses opencv-contrib-python 4.11.0.86.

Safety note: The current source sets `pyautogui.FAILSAFE = False`. This disables PyAutoGUI's corner-based emergency stop. For safer use, consider setting it to `True` and keeping the mouse-control area clear.

8. Testing and Results

The environment setup was validated by importing OpenCV, MediaPipe, PyAutoGUI, and NumPy in the active virtual environment. The MediaPipe Hands API was also checked directly, confirming that `mp.solutions.hands` is available with MediaPipe 0.10.21. The application is launched with `python main.py`.

Test	Expected behaviour	Validation basis
Package import	All four libraries import without an exception.	Confirmed by terminal output.
MediaPipe Hands API	The hands module is accessible.	Confirmed by direct API check.
Camera startup	A live Gesture Control window appears.	Run-time check on the target PC.
Cursor movement	One extended finger moves the cursor.	Functional gesture test.
Scrolling	Two / three fingers scroll vertically / horizontally.	Functional gesture test.
Click and exit	Close fingertips click; Q exits the application.	Functional gesture test.

No benchmark such as frames per second or recognition accuracy is claimed in this report because no formal performance measurement was supplied. Lighting, camera quality, hand position, and system load can affect responsiveness and detection stability.

9. Limitations

• The current program is configured for one hand only.

• Finger counting relies on simple landmark y-coordinate comparisons and may be affected by hand rotation or camera angle.

• The scroll amount is based on frame-to-frame fingertip movement and may feel sensitive without additional smoothing.

• Cursor coordinates are mapped directly across the full frame, without a configurable active region or calibration step.

• The click gesture uses a fixed 30-pixel threshold, which may behave differently at different camera resolutions.

• The application does not currently include a dedicated pause/resume key or an on-screen landmark skeleton.

10. Future Enhancements

• Add cursor smoothing and configurable sensitivity to improve control stability.

• Introduce a calibration step and adjustable active screen region.

• Use more robust gesture classification that considers multiple landmarks and hand orientation.

• Add a pause/resume gesture and a keyboard emergency-stop option.

• Display hand landmarks and a clearer gesture status indicator for easier debugging.

• Measure frame rate, response latency, and gesture success rate under repeatable test conditions.

• Provide user-adjustable click distance, cooldown, and scrolling sensitivity.

11. Conclusion

The Virtual Mouse Using Hand Gestures project demonstrates a practical application of computer vision for touchless human–computer interaction. By combining OpenCV video processing, MediaPipe hand landmarks, NumPy calculations, and PyAutoGUI mouse automation, the prototype maps hand gestures to common cursor and scrolling operations. The working Python environment and compatible MediaPipe/OpenCV package combination provide a reproducible base for further development. Improvements in calibration, smoothing, gesture robustness, safety controls, and performance measurement can make the system more reliable and comfortable to use.

References

• OpenCV Documentation — https://docs.opencv.org/

• MediaPipe Hands — https://developers.google.com/mediapipe/solutions/vision/hand_landmarker

• PyAutoGUI Documentation — https://pyautogui.readthedocs.io/

• NumPy Documentation — https://numpy.org/doc/